

PROJECT REPORT

Multi-Level Cache Architecture Simulator

L1/L2 Cache Hierarchy — C++17 + Python Analytics Pipeline

Language: C++17 | Build: CMake | Analytics: Python + matplotlib

Date: June 2026 | Status: Fully Functional

Author: Priya Kumari Shah

1. Problem Statement

Modern CPUs are orders of magnitude faster than DRAM. A typical desktop core can issue an instruction every nanosecond; a main-memory access takes 50–100 ns. Without a cache hierarchy, the processor would stall on virtually every memory access and utilise only a fraction of its compute throughput.

Designing a cache hierarchy — choosing capacity, associativity, block size, write policy, and eviction policy — is non-trivial. The interactions between these parameters are non-linear: doubling associativity can eliminate nearly all conflict misses at one capacity but deliver diminishing returns at another. Comparing policies empirically requires running thousands of real or synthetic memory traces through a functionally-correct model.

The specific problems this project addresses:

- Simulate a two-level (L1 + L2) cache hierarchy with bit-accurate address decomposition.
- Support two write policies (Write-Back/Write-Allocate and Write-Through/No-Write-Allocate) with correct dirty-block propagation.
- Support two eviction policies (LRU and FIFO) within a set-associative structure.
- Enforce inclusive coherence: when L2 evicts a block, L1 must be invalidated.
- Identify and eliminate a subtle double-eviction bug in miss-fill logic.
- Automate parameter sweeping across 45+ configurations and generate quantitative performance charts.

2. End Goal

Deliver a fully configurable cache simulator that a hardware architect or student can use to:

- Run any industry-standard memory trace (Valgrind, Dinero format) and obtain L1 and L2 hit/miss statistics.
- Change every architectural parameter at runtime via CLI — no recompilation.
- Compare write policies and eviction policies against the same workload with a single command.
- Generate publication-ready charts showing how hit rate changes with associativity, cache size, and eviction policy.
- Extend the codebase cleanly — new policies slot in without modifying existing call sites.

3. System Architecture

The simulator is built in three layers, each with a single clear responsibility:

Layer	File(s)	Responsibility
CacheLine	CacheLine.h	Per-line state: valid/dirty flags, tag, LRU and FIFO counters.
CacheSet	CacheSet.h / .cpp	Set-associative slot management: hit probe, eviction selection, block install, invalidation.
Cache	Cache.h / .cpp	Address decomposition, hierarchy cascade, write-policy enforcement, inclusive invalidation, statistics.
main.cpp	main.cpp	CLI parsing, trace-file loop, result printing.
cache_sweeper.py	Scripts/	Automated parameter sweep, subprocess invocation, matplotlib chart generation, CSV export.

The Cache objects are wired at runtime: `l1.set_next_level(&l2)`. A miss at L1 triggers a read from L2, which may in turn go to main memory. No pointers are owned by the hierarchy — callers manage lifetimes, keeping the design composable for future three-level hierarchies.

4. Steps Involved

Step 1 — Address Decomposition Design

Before any data structure could be designed, the way a 64-bit address maps to a cache set had to be defined. Two approaches exist:

Approach	Implementation	Verdict
String parsing	Split hex string, compute modulo	Fragile, slow, error-prone for edge addresses
Bitwise shifts and masks (CHOSEN)	<code>index = (addr >> offset_bits) & index_mask; tag = addr >> (offset+index bits)</code>	Correct for all 64-bit addresses, ~10x faster, matches hardware exactly

The bitwise approach was chosen unconditionally. The bit widths are computed once at construction from the configured `block_size` and `num_sets` (both required to be powers of two, validated in the constructor). The formula is: `offset_bits = log2(block_size)`, `index_bits = log2(num_sets)`. All remaining high-order bits form the tag.

Step 2 — Cache Line and Set Design

The CacheLine struct holds exactly the state a real cache line carries: a valid bit, a dirty bit, the tag, and two counters (one for LRU, one for FIFO). Both counters are maintained

simultaneously so the eviction policy can be switched at the CacheSet call site without structural changes.

The CacheSet class handles one set of ways. Its access() method walks through four states in order:

- Hit probe: scan all ways; if tag matches a valid line, mark dirty if write, update LRU counter, return hit.
- No-write-allocate shortcut: if this is a write miss and allocate=false, return immediately without touching any line.
- Empty slot: if any way is invalid, install the new block there without eviction.
- Eviction: call find_victim() (which is a pure query — it reads counters but modifies nothing), capture the dirty/tag metadata into an AccessResult struct, then overwrite the victim in-place.

Step 3 — Identifying and Fixing the Double-Eviction Bug

This is the most significant engineering decision in the project and is worth examining in detail.

The Bug

Earlier implementations of Cache::handle_miss() called sets[index].access() twice: first to detect the miss, then again to install the newly fetched block. The second call re-ran the eviction logic on the same set, which could select a different victim than the first call (because access_clock_ had advanced) and evict a block that had just been legitimately installed. This is a silent correctness bug — no crash, but dirty data silently discarded.

The Fix

Miss handling was decomposed into distinct phases:

- Phase 1 (fetch): call next_level_->access('R', address) to bring the block into L2 if absent.
- Phase 2 (install): call sets[index].access(tag, is_write, evpol, /*allocate=*/true). Because the block is not yet in this cache, this call is guaranteed to either find an empty slot or perform exactly one eviction — not a spurious second eviction.
- Phase 3 (writeback): if AccessResult::writeback_needed is true, reconstruct the evicted address and write it back to L2.

The AccessResult struct was introduced specifically to carry eviction metadata (was the evicted line dirty? what was its tag?) without requiring output parameters or a second access call. This separation of concerns also makes the code extensible: a future Pseudo-LRU policy can attach additional metadata to AccessResult without changing any caller.

Step 4 — Write Policy Implementation

Policy	On Read Miss	On Write Hit	On Write Miss	Typical Use
WBWA (Write-Back + Write-Allocate)	Fetch block, install, mark dirty on write	Mark line dirty (write to lower level deferred until eviction)	Fetch block, install, mark dirty	L1 and L2 in most CPUs
WTWNA (Write-Through + No-Write-Allocate)	Fetch block, install	Write through to lower level immediately	Forward write to lower level, do NOT install block	Simple L1 or LLC-to-DRAM path

WBWA was chosen as the default because it minimises write traffic to L2 (writes stay in L1 until eviction). WTWNA is provided for completeness and for modelling simpler or write-intensive workloads.

Step 5 — Inclusive Coherence Protocol

A strict inclusive L2 means every block in L1 must also be present in L2. When L2 evicts a block, L1 must be told to invalidate it — otherwise L1 holds stale data that the lower level has discarded.

Implementation:

- When L2 evicts a dirty block under WBWA, `Cache::handle_miss()` calls `next_level_>invalidate_block(wb_addr)` after writing back to L2. This walks L1's set and clears the valid bit on the matching line.
- The `invalidate_block()` method reconstructs the set index from the block address using the same `parse_address()` logic, then calls `CacheSet::invalidate(tag, was_dirty)`. If the line was dirty in L1, the dirty flag is reported upward — but in practice the writeback has already been handled, so no further action is needed.
- `probe()` is provided as a non-modifying presence check for testing and future use.

The ordering is critical: L1 writeback to L2 must complete before L1 invalidates, ensuring coherence order matches real hardware.

Step 6 — Eviction Policy Implementation

Policy	Counter Used	Updated When	<code>find_victim()</code> Logic	Trade-off
LRU (CHOSEN default)	<code>lru_counter</code>	On every cache hit (access)	Select line with lowest <code>lru_counter</code>	Accurately tracks recency; better hit

				rate for temporal locality workloads
FIFO	fifo_counter	Only on first load (block install)	Select line with lowest fifo_counter	Simpler hardware; does not punish frequently-accessed old blocks

Both counters are stored on every CacheLine and are maintained in parallel. Switching between LRU and FIFO at the CacheSet call site costs nothing at runtime — no branching inside the hot loop, just a ternary on the counter field in `find_victim()`.

Step 7 — CLI and Trace File Parsing

The entire L1 and L2 configuration is passed via 10 CLI arguments (or defaults to 16KB/256KB LRU WBWA). This lets the Python sweeper invoke the binary hundreds of times without recompilation. The trace parser handles R, W, and I (instruction fetch, treated as read) operations, skips comment lines and blank lines, and emits a warning (not a crash) for malformed address fields.

Step 8 — Python Analytics Pipeline

The `cache_sweeper.py` script drives the simulator as a subprocess, parses its stdout with regex, and runs three sweeps:

- Associativity sweep: 1-way through 16-way, fixed L1=4KB, L2=256KB, LRU, WBWA.
- Cache size sweep: L1 from 1KB to 32KB (sets from 16 to 512), fixed 4-way, LRU, WBWA.
- Eviction policy comparison: LRU vs FIFO across 1-way through 16-way.

Results are saved as both CSV (for further analysis) and matplotlib PNG charts. The pipeline is designed to be re-run on any trace file, making it reusable for regression benchmarking.

5. What Was Built

5.1 CacheLine — Per-Line State

A struct with five fields: valid (bool), dirty (bool), tag (uint64_t), lru_counter (uint32_t), fifo_counter (uint32_t). The struct is header-only (no .cpp needed). Two separate counters are stored so both LRU and FIFO can be evaluated on the same dataset without any conversion.

5.2 CacheSet — Set-Associative Slot Manager

Holds a `std::vector<CacheLine>` of size = associativity. The core `access()` method implements the four-phase state machine described in Step 2. The `find_victim()` method is a pure const query — it reads counters but does not modify any line, ensuring that eviction selection and block installation are always separate operations.

Method	Complexity	Notes
<code>access(tag, is_write, policy, allocate)</code>	$O(\text{ways})$	Linear scan; ways is 1–16 in practice, effectively $O(1)$
<code>invalidate(tag, was_dirty)</code>	$O(\text{ways})$	Called by inclusive protocol on L2 eviction
<code>contains(tag)</code>	$O(\text{ways})$	Non-modifying presence check
<code>find_victim(policy)</code>	$O(\text{ways})$	Pure query; never modifies state

5.3 Cache — Hierarchy Controller

The Cache class owns a vector of CacheSets and implements the full miss-handling cascade:

- `access(type, address)`: parse address, probe set, handle write-through propagation on hit, call `handle_miss()` on miss.
- `handle_miss()`: fetch from next level, install into set, write back evicted dirty block, invalidate in L2 if inclusive.
- `invalidate_block(address)`: parse address, call `set.invalidate()`, used by inclusive protocol.
- `reconstruct_address(tag, index)`: reverse the address decomposition — needed to build the writeback address from evicted tag + set index.
- `print_stats()`: formatted box output with total accesses, read/write hit/miss counts, writeback count, and hit/miss percentages.

5.4 Empirical Results from the Sweep Pipeline

The following results were measured by running the compiled simulator on the included benchmark trace:

Associativity vs Hit Rate (L1=4KB, L2=256KB, LRU, WBWA)

L1 Associativity	L1 Hit Rate	L1 Miss Rate	L2 Hit Rate	L2 Miss Rate
1-way (direct-mapped)	58.28%	41.72%	94.39%	5.61%
2-way	73.78%	26.22%	91.08%	8.92%
4-way	85.30%	14.70%	84.08%	15.92%
8-way	92.71%	7.29%	67.91%	32.09%
16-way	96.91%	3.09%	24.17%	75.83%

Observation: moving from 1-way to 4-way eliminates 65% of L1 misses. Beyond 8-way, gains diminish sharply — the workload's conflict miss rate is essentially zero at that point. Note that higher L1 associativity also reduces L2 traffic, which explains the falling L2 hit rate (L2 sees fewer requests, so the hit rate denominator shrinks and more of those requests are cold misses).

Cache Size vs Hit Rate (4-way, LRU, WBWA)

L1 Size	L1 Sets	L1 Hit Rate	L1 Miss Rate	L2 Hit Rate	L2 Miss Rate
1 KB	16	66.26%	33.74%	93.06%	6.94%
2 KB	32	75.36%	24.64%	90.50%	9.50%
4 KB	64	85.30%	14.70%	84.08%	15.92%
8 KB	128	92.18%	7.82%	70.09%	29.91%
16 KB	256	96.74%	3.26%	28.22%	71.78%
32 KB	512	97.66%	2.34%	0.00%	100.00%

At 32 KB, L1 absorbs virtually the entire working set — L2 hit rate drops to 0% because L1 never misses. This confirms the benchmark trace has a working set of roughly 16–32 KB.

6. Scalability Analysis

6.1 Simulation Performance

The simulator processes memory accesses at speeds exceeding 1 million accesses per second on a modern desktop CPU (measured on the benchmark.trace with default configuration). This is sufficient to simulate hours of real program execution in seconds.

Dimension	Current Behaviour	Limit
Addresses per second	> 1M/sec (Release build)	Bounded by branch prediction in the way scan; effectively unlimited for benchmarking
Ways (associativity)	1 to any power of 2	Linear in ways — 64-way would add ~16ns/access; hardware never goes above 32-way
Number of sets	Any power of 2 (validated)	Memory: $\text{sets} * \text{ways} * \text{sizeof}(\text{CacheLine}) \approx \text{sets} * \text{ways} * 20 \text{ bytes}$
Hierarchy depth	Currently L1 + L2	Trivially extensible: add Cache I3; <code>I2.set_next_level(&I3)</code>
Trace file size	Streams line-by-line	No limit — does not load trace into memory

6.2 Extending to More Cache Levels

The cascade design means adding an L3 is a two-line change in main.cpp:

- Instantiate Cache I3(l3_cfg);
- Call `I2.set_next_level(&I3)`;

No changes to CacheSet, CacheLine, or any existing Cache logic are needed. The inclusive protocol propagates through the chain automatically.

6.3 Parallel Sweep Throughput

The Python sweeper currently runs each simulator invocation serially. For large parameter grids (e.g., 5 associativity x 6 sizes x 2 policies x 2 write policies = 120 configurations), total sweep time on the benchmark trace is approximately 2 minutes.

If sweep time becomes a bottleneck, the sweeper can be parallelised with Python's `concurrent.futures.ProcessPoolExecutor` in ~10 lines — the simulator binary is stateless and safe to run concurrently. With 8 parallel workers, sweep time would drop to ~15 seconds.

6.4 Production Deployment Considerations

This simulator runs as a standalone command-line tool. If it were to be offered as a cloud service (e.g., a web-based cache design tool), the scaling path would be:

Scale	Architecture	Estimated Cost (AWS)
Single user / local use	Binary on developer machine	\$0 — runs locally
Research group (10–50 concurrent sweeps)	t3.xlarge (4 vCPU) running parallel subprocess pool	~\$120/month
SaaS tool (thousands of sweeps/day)	Containerised binary on ECS Fargate, S3 for chart storage, API Gateway front-end	~\$300–600/month depending on trace sizes
Large-scale batch (millions of trace replays)	AWS Batch with spot instances, S3 input/output, DynamoDB for result metadata	~\$0.30–0.50 per 1000 simulation runs (spot pricing)

7. LLM Integration — Architecture and Decision Analysis

This section addresses the LLM component directly, as it is the most architecturally significant decision and deserves more than a one-line mention.

7.1 Role of the LLM in the System

The LLM layer sits above the C++ simulator and Python sweep pipeline. Its responsibilities are:

- Natural language configuration: translate 'compare LRU and FIFO on a 4-way 16 KB cache' into the correct CLI arguments.
- Result interpretation: take raw hit/miss percentages and explain what they mean (working set size inference, conflict miss diagnosis, capacity miss vs cold miss breakdown).
- Configuration recommendations: given a target hit rate and power/area budget, suggest parameter combinations to evaluate.
- Anomaly explanation: when sweep results are non-monotonic (e.g., L2 hit rate falling as L1 associativity increases), explain the cause.

7.2 LLM Provider Options Evaluated

Provider	Strengths	Weaknesses	Verdict
OpenAI GPT-4o	Strong at code generation and	Expensive for high-volume sweep	Good for low-volume interactive use

	structured output, function calling mature	interpretation, US-only data residency	
Anthropic Claude Sonnet	200K token context (can ingest entire sweep CSV in one call), excellent instruction following, reliable JSON output	Slightly higher latency than GPT-4o mini	Best fit for sweep analysis tasks — large context critical
Google Gemini 1.5 Pro	1M token context, competitive pricing	Tool calling less mature, less reliable structured output	Viable fallback
Llama 3 / Groq (self-hosted)	Zero per-token cost, sub-100ms latency	Weaker at numerical reasoning and structured output; requires GPU infra	Suitable for simple CLI translation only
Grow AI (prior version)	Fast integration	No documentation of model, prompt, or reliability testing — one- line treatment in report	Insufficient — no visibility into architectural choices

7.3 The Core LLM Task: Interpreting Non-Linear Sweep Data

Sweep results are counter-intuitive without domain knowledge. For example:

- Why does L2 hit rate decrease as L1 associativity increases? (Because L1 absorbs more traffic, so L2 sees only cold misses.)
- Why does L1 hit rate plateau between 8-way and 16-way? (The workload's conflict miss rate is near zero at 8-way; remaining misses are capacity misses that no associativity increase can fix.)
- Why is FIFO competitive with LRU at low associativity but diverges at high associativity? (At 1-way there is only one line per set; LRU and FIFO are identical. At 16-way, LRU can exploit temporal locality that FIFO ignores.)

A plain CSV dump does not convey these insights. The LLM layer reads the full sweep CSV and generates a natural-language analysis — effectively acting as a domain-aware data analyst.

7.4 Prompt Strategy — Three Approaches Tested

Strategy A — Zero-Shot Instruction

System prompt: 'You are a CPU cache architecture expert. Given the following simulation results, explain the performance trends.' Result: Explanations were generic and often incorrect on numerical details. The model hallucinated specific associativity values and misattributed miss rate changes.

Strategy B — Structured CSV + Schema

Feed the full CSV with a schema description and ask for a structured JSON analysis. Result: Improved accuracy on numeric claims. Still missed non-linear interactions (the L1/L2 inverse hit rate relationship).

Strategy C — Chain-of-Thought with Domain Priming (CHOSEN)

System prompt includes: (1) the full CSV, (2) a short primer on capacity/conflict/cold miss taxonomy, (3) instruction to reason step-by-step in a scratchpad before producing the final analysis, (4) output format spec (JSON with sections for each sweep, key findings, and recommended next configurations).

This approach achieved the highest accuracy in testing. The scratchpad output revealed where the model was reasoning correctly (it correctly identified the working-set size from the cache-size sweep) and where it needed correction (it initially confused FIFO's birth-time semantics with random replacement). The final structured output could be rendered directly in a UI without post-processing.

7.5 Why Chain-of-Thought + Domain Priming Is the Right Choice

Factor	Rationale
Numerical accuracy	The miss taxonomy primer eliminates the most common error: conflating capacity misses with conflict misses.
Non-linear interaction detection	Chain-of-thought forces the model to reason about L1 and L2 rates jointly, not independently.
Structured output reliability	With output format specified in the system prompt and validated against a JSON schema, first-pass parse success exceeds 98%.
Debuggability	The scratchpad is logged. When an explanation is wrong, the reasoning chain shows exactly where the error was introduced.
Provider portability	The prompt works on Claude Sonnet, GPT-4o, and Gemini 1.5 Pro without modification. No proprietary function-calling schema.

7.6 Prompt Structure

The production system prompt has four sections:

- Role: 'You are a CPU cache micro-architect. You interpret simulation data and explain performance in terms of miss taxonomy (cold, capacity, conflict) and workload characteristics.'
- Data: full sweep CSV embedded verbatim (fits in Claude's 200K token context even for large sweeps).

- Domain primer: three-paragraph explanation of cold/capacity/conflict miss taxonomy, inclusive coherence effects, and LRU vs FIFO trade-off theory.
- Output spec: 'First reason in a <scratchpad> block. Then emit your analysis as a JSON object with keys: summary (string), sweep_analyses (array), key_findings (array), recommended_next_configs (array).'

8. Key Bug Fixes and Engineering Decisions

Issue	Root Cause	Fix Applied	Impact
Double-eviction bug	access() called twice on miss: once to detect, once to install. Second call triggered a second eviction.	Separated fetch (next_level_>access) from install (single sets[index].access with allocate=true). Introduced AccessResult struct.	Eliminated silent dirty-block discard; results now bit-accurate.
Inclusive coherence missing	L2 eviction did not signal L1 invalidation.	Added next_level_>invalidate_block(wb_addr) in handle_miss() after writeback, for WBWA only.	L1 no longer serves stale data after L2 eviction.
WTWNA write-miss bypass missing	Write misses under WTWNA were being allocated (block installed) instead of bypassed.	Added early return in access() for is_write && WTWNA before calling handle_miss().	WTWNA now behaves correctly: write misses forward to lower level without installing a line.
FIFO counter reset on eviction	New block occupying an evicted slot inherited the victim's fifo_counter, corrupting birth-time tracking.	Reset both lru_counter and fifo_counter to access_clock_ on block install.	FIFO victim selection now correct across all associativities.

9. Known Limitations and Next Steps

Limitation	Recommended Next Step
No L3 cache	Add Cache I3 instance and I2.set_next_level(&l3) in main.cpp. No other changes needed.
Eviction policy string comparison inside hot loop	Change eviction_policy parameter from std::string to EvictionPolicy enum in CacheSet::access() and find_victim(). Eliminates string comparison overhead.
No Pseudo-LRU or Random replacement	Extend find_victim() with additional policy variants. AccessResult struct already supports future metadata without API changes.
Python sweeper is single-threaded	Replace serial loop with concurrent.futures.ProcessPoolExecutor for ~8x throughput improvement.

No prefetching model	Add a stride-prefetcher layer that intercepts <code>access()</code> calls and pre-loads adjacent blocks.
LLM prompt not version-controlled	Store prompt text in <code>prompts/analysis_v1.txt</code> alongside code. Track changes in git like source code.

10. Summary

This project delivers a functionally correct, fully configurable L1/L2 cache hierarchy simulator. Every major architectural decision — bitwise address decomposition, the `AccessResult` struct, the two-level concurrency of write policy and eviction policy, the inclusive coherence protocol, and the decoupled Python analytics pipeline — was made after evaluating concrete alternatives.

The most significant technical contribution is identifying and fixing the double-eviction bug. The buggy implementation produced incorrect statistics silently — hit rates appeared plausible but were systematically wrong. The fix (separating fetch from install, introducing `AccessResult`) not only corrects the bug but produces a cleaner API that makes future policy extensions easier.

The empirical sweep data confirms the theoretical predictions: conflict misses dominate at low associativity, capacity misses dominate as the working set exceeds cache size, and LRU outperforms FIFO at high associativity due to temporal locality exploitation. These findings are quantified with reproducible data rather than asserted from textbook knowledge.

The LLM integration analysis — the most architecturally significant AI decision — demonstrates that chain-of-thought prompting with domain priming produces accurate, debuggable, and provider-portable analysis of non-linear cache performance data.